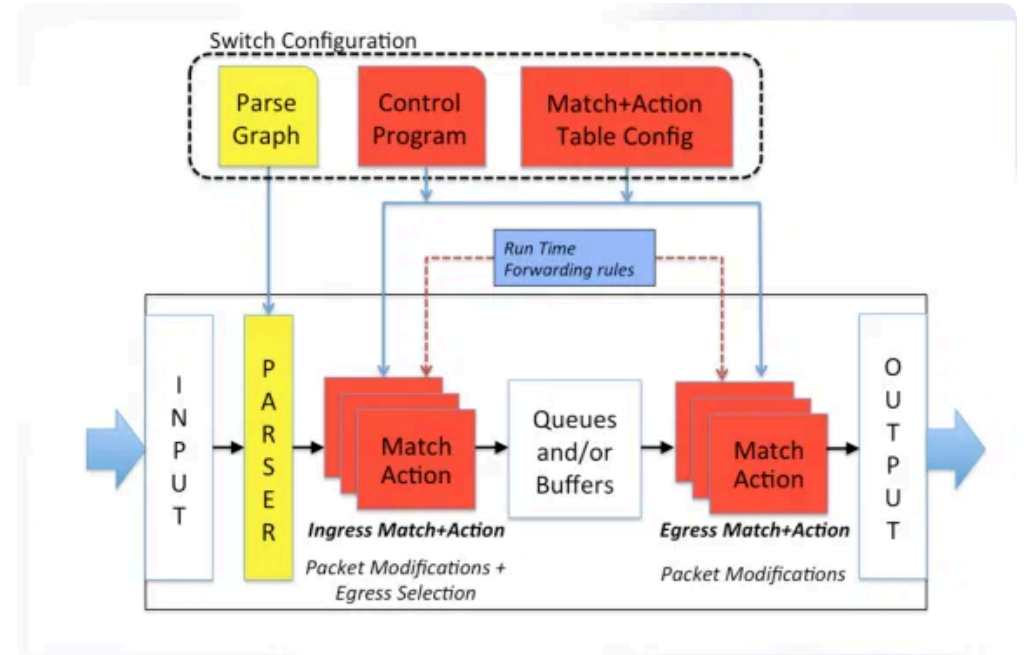


Introdução ao P4

P4 (Programming Protocol-independent Packet Processors) é uma linguagem de domínio específico para programação de dispositivos de rede.

Desenvolvida para permitir que engenheiros de rede e desenvolvedores de aplicações programem o comportamento do plano de dados de dispositivos de rede, como switches, roteadores e NICs.

- ✓ Independência de protocolo
- ✓ Programabilidade do plano de dados
- ✓ Flexibilidade e inovação rápida
- ✓ Controle total sobre o processamento de pacotes



Arquitetura P4

A arquitetura P4 define como os pacotes são processados em dispositivos de rede programáveis. Os principais componentes são:

▼ Parser

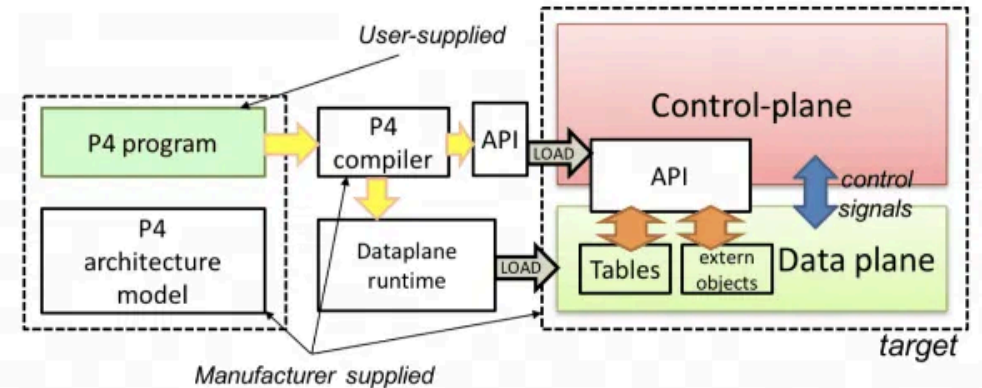
Responsável por extrair campos de cabeçalho dos pacotes de entrada, identificando protocolos e estruturas de dados.

⚙️ Match-Action Pipeline

Núcleo do processamento, onde tabelas de correspondência determinam ações a serem aplicadas aos pacotes com base nos campos extraídos.

📦 Deparser

Reconstrói os pacotes após o processamento, recompondo os cabeçalhos modificados antes de enviar o pacote para a saída.



Fundamentos da Linguagem P4

A linguagem P4 possui conceitos fundamentais que permitem a programação do plano de dados de dispositivos de rede:

Tipos de Dados

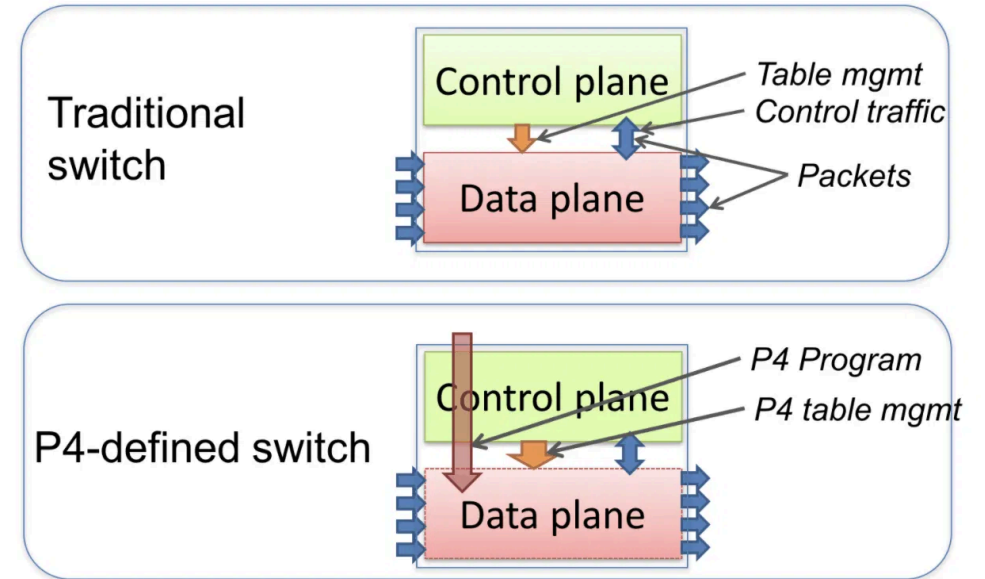
P4 oferece tipos de dados básicos como `bit<n>`, `int<n>`, `varbit<n>` e tipos derivados.

Cabeçalhos

```
header ethernet_t {  
  bit<48> dstAddr;  
  bit<48> srcAddr;  
  bit<16> etherType;  
}
```

Tabelas e Ações

Tabelas definem regras de correspondência e ações a serem executadas quando há correspondência.



BMv2 e Mininet

BMv2 (Behavioral Model version 2)

Implementação de referência em software do switch P4, permitindo testar programas P4 sem hardware específico.

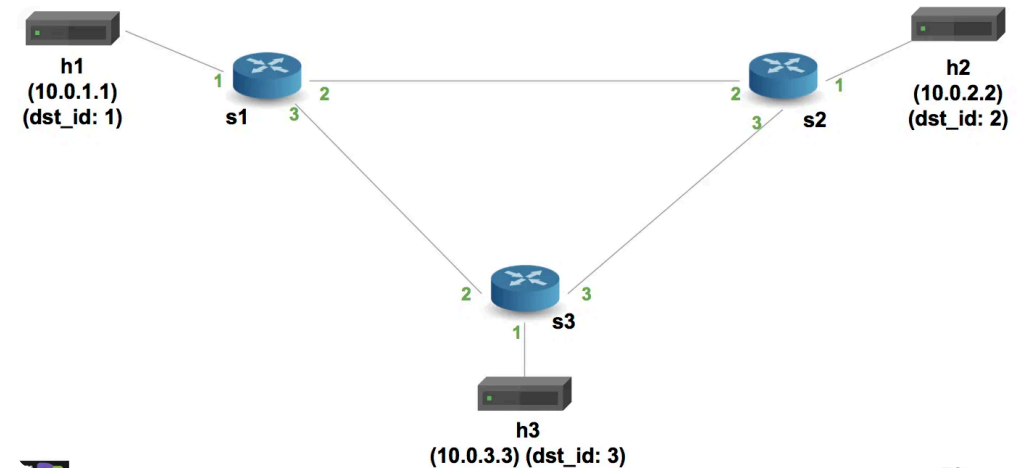
- Escrito em C++17
- Suporta diferentes arquiteturas P4
- Inclui ferramentas de depuração

Mininet

Emulador de rede que cria uma rede virtual realista em um único computador, ideal para testes e desenvolvimento.

Integração BMv2-Mininet

Permite criar topologias de rede virtuais com switches P4 programáveis para testar e validar programas P4.



Ambiente de Desenvolvimento

Para desenvolver programas P4 e testá-los com BMv2 e Mininet, é necessário configurar um ambiente adequado:

📥 Instalação das Ferramentas

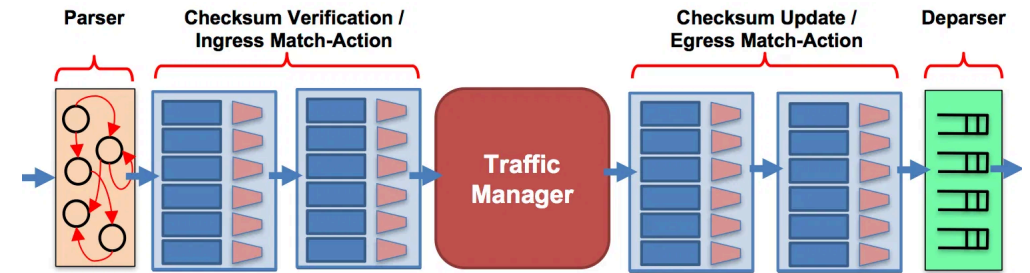
- Compilador P4 (p4c)
- BMv2 (behavioral-model)
- Mininet
- P4Runtime

</> Compilação de Programas P4

```
p4c-bm2-ss --p4v 16 -o programa.json programa.p4
```

▶ Execução com Mininet

Criação de topologias personalizadas com switches BMv2 e execução de programas P4 compilados.



Programação Básica em P4

Vamos implementar um exemplo básico de encaminhamento IPv4 em P4:

</> Ação de Encaminhamento

```
action ipv4_forward(macAddr_t dstAddr, egressSpec_t port)
{
    standard_metadata.egress_spec = port;
    hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
    hdr.ethernet.dstAddr = dstAddr;
    hdr.ipv4.ttl = hdr.ipv4.ttl - 1;
}
```

📊 Tabela de Encaminhamento

```
table ipv4_lpm {
    key = {
        hdr.ipv4.dstAddr: lpm;
    }
    actions = {
        ipv4_forward;
        drop;
        NoAction;
    }
    size = 1024;
    default_action = drop();
}
```

The screenshot shows the Ganache application window with the 'SERVER' tab selected. The interface includes a top navigation bar with tabs for WORKSPACE, SERVER, ACCOUNTS & KEYS, CHAIN, ADVANCED, and ABOUT. On the right side of the top bar are 'CANCEL' and 'RESTART' buttons. The main content area is divided into sections for SERVER, NETWORK ID, AUTOMINE, ERROR ON TRANSACTION FAILURE, and CHAIN FORKING. The SERVER section has a 'HOSTNAME' dropdown menu with options: '10.0.2.15 - eth0' (selected), '0.0.0.0 - All Interfaces', '127.0.0.1 - lo', and '10.0.2.15 - eth0 / 545'. To the right of the dropdown is a text description: 'The server will accept RPC connections on the following host and port.' The NETWORK ID section has a text input field containing '5777' and a description: 'Internal blockchain identifier of Ganache server.' The AUTOMINE section has a toggle switch that is currently turned off, with a description: 'Process transactions instantaneously.' The ERROR ON TRANSACTION FAILURE section has a toggle switch that is currently turned off, with a description: 'When transactions fail, throw an error. If disabled, transaction failures will only be detectable via the "status" flag in the transaction receipt. Disabling this feature will make Ganache handle transaction failures like other Ethereum clients.' The CHAIN FORKING section has a greyed-out area with a warning icon and text: 'Forking can only be updated when creating a new workspace.' Below this, it says 'Forking is Disabled'.

Recursos Intermediários do P4

Após dominar os conceitos básicos, podemos explorar recursos mais avançados do P4:

Tunneling e Encapsulamento

Implementação de protocolos de encapsulamento como VXLAN, GRE ou MPLS para criar túneis de rede.

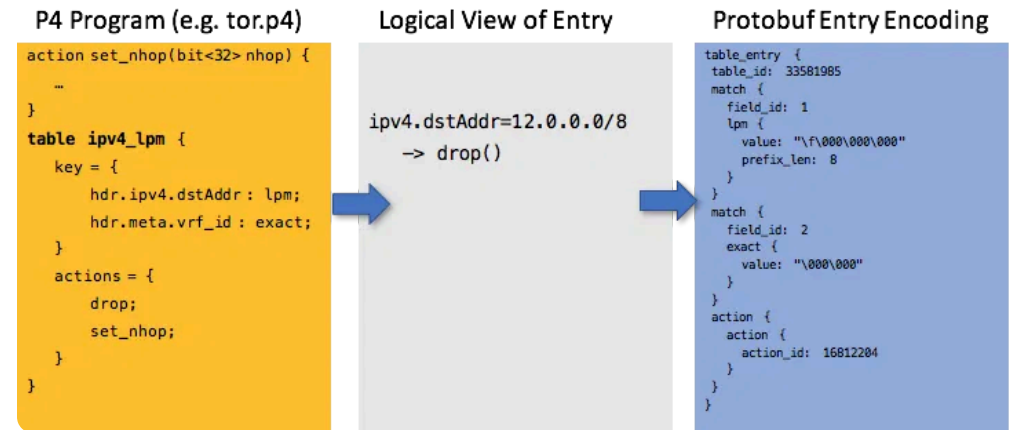
Multicast

Configuração de grupos multicast para enviar pacotes para múltiplos destinos simultaneamente.

```
// Exemplo de configuração multicast
action set_multicast_group(MulticastGroupId_t gid) {
  standard_metadata.mcast_grp = gid;
}
```

Controle de Fluxo e QoS

Implementação de mecanismos de qualidade de serviço, como filas de prioridade e controle de congestionamento.



Programação Avançada em P4

Técnicas avançadas de programação em P4 permitem implementar funcionalidades sofisticadas em dispositivos de rede:

📈 Telemetria In-band Network (INT)

Coleta de informações detalhadas sobre o desempenho da rede diretamente no plano de dados, permitindo monitoramento em tempo real.

⚖️ Balanceamento de Carga

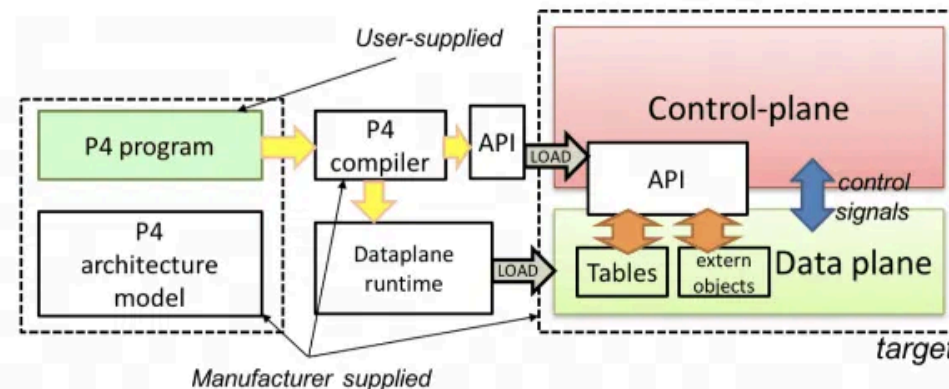
Implementação de algoritmos de balanceamento de carga como Equal-Cost Multi-Path (ECMP) ou algoritmos baseados em hash consistente.

🛡️ Segurança e Firewall

Criação de regras de firewall e mecanismos de segurança diretamente no plano de dados para proteção contra ataques.

⚙️ P4Runtime

API para comunicação entre o plano de controle e o plano de dados, permitindo configuração dinâmica de dispositivos P4.



Casos de Uso e Aplicações

A linguagem P4 tem sido aplicada em diversos cenários reais, demonstrando sua versatilidade e potencial:

Monitoramento de Rede

Implementação de sistemas de monitoramento em tempo real que coletam estatísticas diretamente no plano de dados, reduzindo a sobrecarga no plano de controle.

Balanceamento de Carga

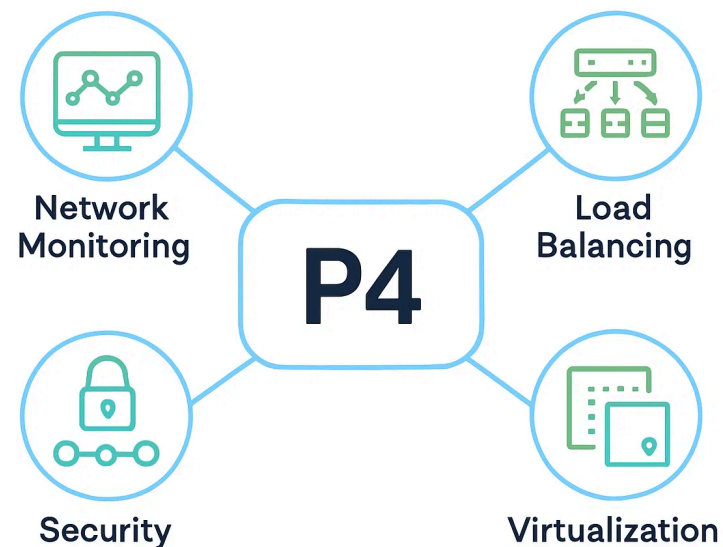
Distribuição eficiente de tráfego entre múltiplos servidores, utilizando algoritmos personalizados implementados diretamente nos switches.

Segurança

Deteção e mitigação de ataques DDoS e outras ameaças de segurança diretamente no plano de dados, permitindo respostas mais rápidas.

Virtualização de Rede

Criação de redes virtuais isoladas sobre infraestrutura física compartilhada, com controle granular sobre o processamento de pacotes.



Projeto Prático

Para aplicar os conhecimentos adquiridos durante o curso, propomos o desenvolvimento de um projeto prático:

📋 Objetivo

Implementar um sistema de balanceamento de carga em uma topologia de rede usando P4 e Mininet com switches BMv2.

📋 Etapas

1. Criar uma topologia com múltiplos hosts e switches P4
2. Implementar um programa P4 para encaminhamento básico
3. Adicionar funcionalidade de balanceamento de carga
4. Implementar monitoramento de estatísticas
5. Testar e validar o funcionamento

🎓 Aprendizado

Este projeto permitirá aplicar conceitos de programação P4, integração com Mininet, manipulação de cabeçalhos e implementação de algoritmos de rede.

